Python 程序设计语言



第四部分 控制结构

if选择结构、 for循环结构、 while循环结构

布尔值

- 在这个世界上，文字和认知中有真和假的判断，而对于计算机而言，布尔值 True 和 False 就表示 真 和 假

```
True
False
```

- True 、 False 是比较显式的真和假, 而在Python中以下值都会被看作是假（False）：

```
False None 0 "" () [] {}
```

== 符号

- == 符号是判断两个值、变量之间是否相等的操作符，相等则返回布尔值 True

```
1 == True
```

```
True
```

```
0 == False
```

```
True
```

- 对于字符型变量也是可以这么操作的：

```
pokemon_name = 'Pikachu'

pokemon_name == 'Pikachu'
```

```
True
```

```
pokemon_name == ''
```

```
False
```

if 选择结构

```
if 判断条件:
    执行语句
```

- 判断条件为真（True）的时候才执行冒号后下面的语句
- 比如现在我们已有一个精灵宝贝的 HP 值，而我只希望当这个 HP 值大于 20 的时候才打印出来

```
HP=30
if HP>20:
   print("此精灵宝贝的HP值大于20")
```

此精灵宝贝的HP值大于20

基本的条件语句

- 一个基本的条件语句由一个关键字if开头，跟上一个表示条件的逻辑表达式，然后是一个冒号:。
- 从下一行开始，所有缩进了的语句就是当条件成立（逻辑表达式计算的结果为True）的时候要执行的语句。
- 如果条件不成立，就跳过这些语句不执行，而继续下面的其他语句。

```
x = int(input())
y=z=0
if x>20:
    y = 100          # 书写缩进，当x>20时执行
    z = 200          # 书写缩进，当x>20时执行
print(y+z)           # if语句后续的语句
```

嵌套的条件语句

- 分支语句（块）中包含另一个if语句，这种情况称为条件语句的嵌套
- if code == 'R':

```
if count <20:
    print('一切正常')
else:
    print('继续等待')
```

连缀的if-elif-else

- 分段函数在数学中也是常见的，比如下面的这个函数：
- f(x) = -1; x<0
  f(x) = 0; x=0
  f(x) = 2x; x>0

- 例 分段函数
- ```
  x = int(input())
  f = 0
  if x < 0:
      f = -1
  elif x == 0:
      f = 0
  else:
      f = 2 * x
  print(f)
  ```
- 2次条件判断，实现3个分支。
  ```

for 循环结构

```
for 元素变量 in 序列:
    执行语句
```

- for 循环是可以依次得到序列循环中每个元素，并依次处理

```
pokemon=['Pikachu',"Bulbasaur",'Squirtle'] #精灵宝贝存放列表
for name in pokemon:
        print(name)
```

Pikachu
Bulbasaur
Squirtle

- 现在有一个列表，这个列表存放的是 10 个小精灵的 HP 数值，现在希望得到里面每个数字都乘以 2

```
HP=[110,103,103,75,85,105,50,75,105,210,75,45,55,75]
HP_new=[]
for n in HP:
    HP_new.append(n*2)
print(HP_new)
```

[220, 206, 206, 150, 170, 210, 100, 150, 210, 420, 150, 90, 110, 150]

for 循环结构

求[m,n]之间的素数和(C语言思路)

```
sum = 0
m,n = map(int, input().split())
if m==1:          # 1不是素数
    m = 2
prime = []        # 记录已知素数的列表

for x in range(2, n+1):
    for k in prime:
        if x%k == 0:
            break
    else:
        if x>=m:
            sum += x
            prime.append(x)          # 加入已知素数的列表，
            # 用于下一次计算
print(sum)
```

（如果for循环正常结束，else中语句执行。如果是break的，则不执行。）

for 循环结构

row代表列表lst的某个元素，本身又是一个列表。col代表row列表中的某一个元素。

```
lst=[[4,71],[58,114,94,2],[67,6,45]]
for row in lst:
    for col in row:
        print(col,end=' ')
    print()
```

运行结果：

```
4 71
58 114 94 2
67 6 45
```

while 循环结构

- while循环和for循环不同的是，它的停止条件是个人自己设定的：

```
while 判断条件：
    执行语句
```

- 判断条件和if语句是相同的，而什么时候用while呢？在你确定满足条件而不确定需要的循环次数时，那么while是最好的选择。现在有一个小精灵的HP数值变量，我希望它在大于20的时候，逐次变小，直到等于20为止。

```
HP = 30
while HP>20:
    HP = HP - 1
print HP
```

```
20
```

while 循环结构

- 计算log2x
- x = int(input())

  count = 0

  while x >1:

  ```
  x //= 2
  count += 1
  ```

  print(count)
- 输入：32
- 输出：5
- 输入：31
- 输出：4

步骤	x	count	说明
1	32	0	进入循环之前
2	16	1	循环的第一轮
3	8	2	
4	4	3	
5	2	4	
6	1	5	

步骤	x	count	说明
1	31	0	进入循环之前
2	15	1	循环的第一轮，31/2->15
3	7	2	
4	3	3	
5	1	4	

- 注：单斜杠（/）表示除法，且不管除数和被除数是不是整数，最后结果都是float类型。双斜杠（//）表示地板除，即先做除法（/），然后向下取整（floor），至少有一方是float型时，结果为float型；两个数都是int型时，结果为int型。

循环内的控制

- 跳出循环break
- 猜数游戏
- import random

```
number = random.randint(0,100)
count = 0
while True:    # 循环条件是逻辑常量True，意味着无限循环
    a = int(input('输入你猜的数:'))
    count += 1
    if a == number:
        break   #  跳出当前循环，执行while后面的语句。
    elif a>number:
        print('你猜的大了')
    else:
        print('你猜的小了')
print('猜中了！你用了{}次！'.format(count))
```

循环内的控制

- 跳过一轮循环continue

  continue语句作用是跳过本次循环，进入到下一次循环。

  计算偶数的平均数

  要求：输入一系列的整数，最后输入-1表示输入结束，然后程序计算出这些数字中的偶数的平均数，输出输入的数字中的偶数的个数和偶数的平均数。

```
sum = 0
count = 0
while True:
    number = int(input())
    if number == -1:
        break
    if number % 2 == 1:
        continue    # 如果是奇数的话，跳过后面的循环语句，
        # 进入下一次循环。
    sum += number
    count += 1
average = sum / count
print(average)
```

组合使用

- 除了单独使用上述控制结构之外，我们还可以嵌套使用。比如说在前面的HP序列，我只希望其中的偶数乘以"2"，奇数不变。

```
HP=[110,103,103,75,85,105,50,75,105,210,75,45,55,75]
HP_new=[]
for n in HP:
    if not n%2:
        HP_new.append(n*2)
    else:
        HP_new.append(n)
print(HP_new)
```

[220, 103, 103, 75, 85, 105, 100, 75, 105, 420, 75, 45, 55, 75]

推导式

- 推导式comprehensions（又称解析式），是Python的一种独有特性。
- 推导式是可以从一个数据序列构建另一个新的数据序列的结构体。
- 共有三种推导，在Python2和3中都有支持：
  - ✓ 列表(list)推导式
  - ✓ 字典(dict)推导式
  - ✓ 集合(set)推导式

列表推导式

基本格式

```
variable = [out_exp_res for out_exp in input_list if out_exp == 2]
```

out_exp_res:　　列表生成元素表达式，可以是有返回值的函数。

for out_exp in input_list:　　迭代input_list将out_exp传入out_exp_res表达式中。

if out_exp == 2:　　根据条件过滤哪些值可以。

例一：

```
multiples = [i for i in range(30) if i % 3 is 0] print(multiples)
# Output: [0, 3, 6, 9, 12, 15, 18, 21, 24, 27]
```

例二：

```
def squared(x):
      return x*x
multiples = [squared(i) for i in range(30) if i % 3 is 0]
Print（multiples）
# Output: [0, 9, 36, 81, 144, 225, 324, 441, 576, 729]
```

例三

```
lst = [i**2 for i in range(5) ]
print (lst)
```

```
[0,1,4,9,16]
```

例四，对列表中每个数值逐个减去均值

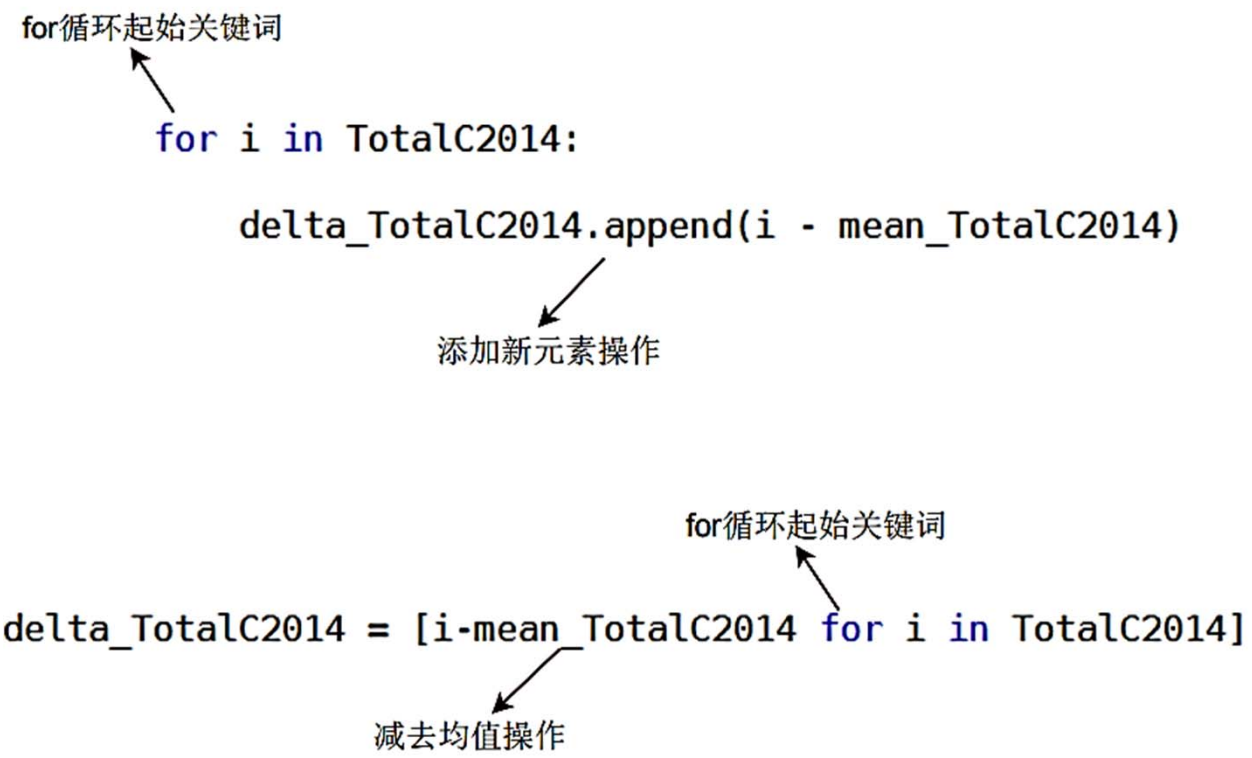
```
TotalC2014 = [603119,1958221,1114409,1422590,7620082,1477177,750019,178929,274013,4121680,2850
              2931630,1132354,1770741,4279443,1437640,1400614,410530,2881550,1912065,1903839,1
              310142,2340188,679136,660256,3742892,3809648,1679856,987145,3902434,204459,16320
              98385210,797995,2430205,139897,2011941,1868867,752942,535612]

mean_TotalC2014 = sum(TotalC2014) / len(TotalC2014)
delta_TotalC2014 = [i-mean_TotalC2014 for i in TotalC2014] # 列表推导式

print delta_TotalC2014
```

```
[-3181342, -1826240, -2670052, -2361871, 3835621, -2307284, -3034442, -3605532, -3510448, 3372
```

- 含义解析：
- 关键词for循环后跟的是循环语法，这部分不变；而在for循环真正表达式部分则在列表推导式中移前，运算结果直接添加入列表中。



例五

```
>>> [(i,j) for i in range(3) for j in range(3)]
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
```

请注意下面语句i是外层循环，j是内层循环

```
>>> [[i+j for i in range(3)] for j in range(3)]
[[0, 1, 2], [1, 2, 3], [2, 3, 4]]
```

字典推导式

- 字典也可以用推导式，但没有列表推导式那么常用。
- 字典推导式多用于需要元素有一一对应关系时，比如前面谈到当变量是字符型时，需要将字符转换为一一对应的数值型。
- 字典推导和列表推导的使用方法是类似的，只不中括号该改成大括号。

例一：大小写key合并

```
mcase = {'a': 10, 'b': 34, 'A': 7, 'Z': 3}
mcase_frequency = {
    k.lower(): mcase.get(k.lower(), 0) + mcase.get(k.upper(), 0)
    for k in mcase.keys()
    if k.lower() in ['a','b']
}
print (mcase_frequency)
#  Output: {'a': 17, 'b': 34}
```

例子二：快速更换key和value

```
mcase = {'a': 10, 'b': 34}

mcase_frequency = {v: k for k, v in mcase.items()}

print （mcase_frequency）

# Output: {10: 'a', 34: 'b'}
```

- United States Energy, Census, and GDP 2010-2014 数据集中Region变量是字符型，如现需要对其进行数值转换。

```
Region = ['West','South','West','South','West','West','Northeast','South','South','South','South','West','Midwest','West',\
          'Midwest','Midwest','Midwest','South','South','Northeast','South','Northeast','Midwest','Midwest','Midwest','South',\
          'West','South','Midwest','Midwest','Northeast','Northeast','West','West','Northeast','Midwest','South','West',\
          'Northeast','Northeast','South','Midwest','South','South','nan','West','South','Northeast','West','Midwest','South','West']

r_t_v = {ch:i for i, ch in enumerate(Region)}  # 字典推导式
v_t_r = {value:key for key, value in r_t_v.items()}  #字典推导式

print r_t_v
print v_t_r
```

```
{'West': 51, 'Northeast': 47, 'Midwest': 49, 'nan': 44, 'South': 50}
{49: 'Midwest', 50: 'South', 51: 'West', 44: 'nan', 47: 'Northeast'}
```

enumerate() 函数用于将一个可遍历的数据对象(如列表、元组或字符串)组合为一个索引序列，同时列出数据和数据下标，一般用在 for 循环当中。

- enumerate 是Python的一个常用内置函数，它用在列表中时，不但会产生列表内的元素，并且会从"0"开始按顺序生成序号。

```
for i, j in enumerate(["a", "b", "c"]):
    print i, j
```

```
0 a
1 b
2 c
```

集合推导式

它们跟列表推导式也是类似的。 唯一的区别在于它使用大括号{}。

```
squared = {x**2 for x in [1, 1, 2]}
print(squared)

# Output: set([1, 4])
```

复杂数据结构

- 在解决实际问题时，还经常需要用到其他复杂的数据结构，如堆、栈、队列、树、图等等。
- 有些结构Python已经提供，而有些则需要自己利用基本数据结构来实现。

堆

```
>>> import heapq                          #heapq和random是Python标准库
>>> import random
>>> data=range(10)
>>> data=list(data)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> random.choice(data)                   #随机选择一个元素
9
>>> random.shuffle(data)                  #随机打乱顺序
>>> data
[6, 1, 3, 4, 9, 0, 5, 2, 8, 7]
>>> heap=[]
>>> for n in data:                        #建堆
    heapq.heappush(heap,n)
>>> heap
[0, 2, 1, 4, 7, 3, 5, 6, 8, 9]
>>> heapq.heappush(heap,0.5)              #入堆，自动重建
>>> heap
[0, 0.5, 1, 4, 2, 3, 5, 6, 8, 9, 7]
>>> heapq.heappop(heap)                   #出堆，自动重建
0
```

堆

```
>>> myheap=[1,2,3,5,7,8,9,4,10,333]
>>> heapq.heapify(myheap)                    #建堆
>>> myheap
[1, 2, 3, 4, 7, 8, 9, 5, 10, 333]
>>> heapq.heapreplace(myheap,6)              #弹出最小元素，同时插入新元素
1
>>> myheap
[2, 4, 3, 5, 7, 8, 9, 6, 10, 333]
>>> heapq.nlargest(3, myheap)                #返回前3个最大的元素
[333, 10, 9]
>>> heapq.nsmallest(3, myheap)               #返回前3个最小的元素
[2, 3, 4]
```

队列

```
>>> import queue          #queue是Python标准库
>>> q=queue.Queue()
>>> q.put(0)              #入队
>>> q.put(1)
>>> q.put(2)
>>> q.queue
deque([0, 1, 2])
>>> q.get()               #出队
0
>>> q.queue               #查看队列中的元素
deque([1, 2])
>>> q.get()
1
>>> q.queue
deque([2])
```

队列

■queue模块还提供了“后进先出”队列和优先级队列。

```
>>> from queue import Queue          #LILO队列
>>> q = Queue()                      #创建队列对象
>>> q.put(0)                         #在队列尾部插入元素
>>> q.put(1)
>>> q.put(2)
>>> print(q.queue)                   #查看队列中所有元素
deque([0, 1, 2])
>>> q.get()                          #返回并删除队列头部元素
0
>>> q.get()
1
```

队列

```
>>> from queue import LifoQueue  #LIFO队列
>>> q = LifoQueue()              #创建LIFO队列对象
>>> q.put(1)                     #在队列尾部插入元素
>>> q.put(2)
>>> q.put(3)
>>> q.queue                      #查看队列中所有元素
[1, 2, 3]
>>> q.get()                      #返回并删除队列尾部元素
3
>>> q.get()
2
>>> q.queue
[1]
>>> q.get()                      #对空队列调用get()方法会阻塞当前线程
```

队列

```
>>> from queue import PriorityQueue   #优先级队列
>>> q = PriorityQueue()                #创建优先级队列对象
>>> q.put(3)                  #插入元素
>>> q.put(8)                  #插入元素
>>> q.put(100)
>>> q.queue                   #查看优先级队列中所有元素
[3, 8, 100]
>>> q.put(1)                  #插入元素，自动调整优先级队列
>>> q.put(2)
>>> q.queue
[1, 2, 100, 8, 3]
>>> q.get()                   #返回并删除优先级最低的元素
1
>>> q.get()                   #请多执行几次该语句并观察返回的数据
2
```

优先队列是0个或多个元素的集合,每个元素都有一个优先权或值,对优先队列执行的操作有1) 查找;2) 插入一个新元素;3) 删除.在最小优先队列(min priority queue)中,查找操作用来搜索优先权最小的元素,删除操作用来删除该元素;对于最大优先队列(max priority queue),查找操作用来搜索优先权最大的元素,删除操作用来删除该元素.优先权队列中的元素可以有相同的优先权,查找与删除操作可根据任意优先权进行.

队列

- Python标准库collections提供了双端队列deque: Deque (Doubly Ended Queue) in Python is implemented using the module “collections“. Deque is preferred over list in the cases where we need quicker append and pop operations from both the ends of container, as deque provides an O(1) time complexity for append and pop operations as compared to list which provides O(n) time complexity.

```
>>> from collections import deque
>>> q = deque(maxlen=5)              #创建双端队列
>>> for item in [3, 5, 7, 9, 11]:    #添加元素
        q.append(item)
>>> q.append(13)                     #队列满，自动溢出
>>> q.append(15)
>>> q
deque([7, 9, 11, 13, 15], maxlen=5)
>>> q.appendleft(5)                  #从左侧添加元素，右侧自动溢出
>>> q
deque([5, 7, 9, 11, 13], maxlen=5)
```

栈

- 栈是一种“后进先出（LIFO）”或“先进后出（FILO）”的数据结构。
- Python列表本身就可以实现栈结构的基本操作。例如，列表对象的append()方法是在列表尾部追加元素，类似于入栈操作；pop()方法默认是弹出并返回列表的最后一个元素，类似于出栈操作。
- 但是直接使用Python列表对象模拟栈操作并不是很方便，例如当列表为空时再执行pop()出栈操作时则会抛出一个不很友好的异常；另外，也无法限制栈的大小。

栈

- 可以直接使用列表来实现栈结构

```
>>> myStack = []
>>> myStack.append(3)
>>> myStack.append(5)
>>> myStack.append(7)
>>> myStack
[3, 5, 7]
>>> myStack.pop()
7
>>> myStack.pop()
5
>>> myStack.pop()
3
>>> myStack.pop()
出错
```

栈

- 封装列表实现栈结构

```
class Stack:
    def __init__(self, size = 10):
        self._content = []              #使用列表存放栈的元素
        self._size = size               #初始栈大小
        self._current = 0               #栈中元素个数初始化为0

    def empty(self):
        self._content = []
        self._current = 0

    def isEmpty(self):
        if not self._content:
            return True
        else:
            return False
```

栈

```
def setSize(self, size):
    #如果缩小栈空间，则删除指定大小之后的已有元素
    if size < self._current:
        for i in range(size, self._current)[::-1]:
            del self._content[i]
        self._current = size
    self._size = size

def isFull(self):
    if self._current == self._size:
        return True
    else:
        return False
```

栈

```
def push(self, v):
    if len(self._content) < self._size:
        self._content.append(v)
        self._current = self._current+1  #栈中元素个数加1
    else:
        print('Stack Full!')

def pop(self):
    if self._content:
        self._current = self._current-1  #栈中元素个数减1
        return self._content.pop()
    else:
        print('Stack is empty!')
```

栈

```
    def show(self):
        print(self._content)

    def showRemainderSpace(self):
        print('Stack can still PUSH ', self._size-self._current, ' elements.')

if __name__ == '__main__':
    print('Please use me as a module.')
```

栈

■自定义栈的用法

```
>>> import Stack
>>> x = Stack.Stack()
>>> x.push(1)
>>> x.push(2)
>>> x.show()
[1, 2]
>>> x.pop()
2
>>> x.show()
[1]
>>> x.showRemainderSpace()
Stack can still PUSH  9  elements.
>>> x.isEmpty()
False
>>> x.isFull()
False
```

链表

- 可直接使用列表来实现：

```
>>> linkTable = []
>>> linkTable.append(3)
>>> linkTable.append(5)
>>> linkTable
[3, 5]
>>> linkTable.insert(1,4)
>>> linkTable
[3, 4, 5]
>>> linkTable.remove(linkTable[1])
>>> linkTable
[3, 5]
```

二叉树

- 使用代码中的类BinaryTree创建的对象不仅支持二叉树的创建以及前序遍历、中序遍历与后序遍历等三种常用的二叉树节点遍历方式，还支持二叉树中任意“子树”的遍历。

```
class BinaryTree:
    def __init__(self, value):
        self.__left = None
        self.__right =  None
        self.__data = value


    def insertLeftChild(self, value):  #创建左子树
        if self.__left:
            print('__left child tree already exists.')
        else:
            self.__left = BinaryTree(value)
            return self.__left
```

二叉树

```
def insertRightChild(self, value): #创建右子树
    if self.__right:
        print('Right child tree already exists.')
    else:
        self.__right = BinaryTree(value)
        return self.__right

def show(self):
    print(self.__data)
```

二叉树

```
def preOrder(self):                 #前序遍历
    print(self.__data)              #输出根节点的值
    if self.__left:
        self.__left.preOrder()      #遍历左子树
    if self.__right:
        self.__right.preOrder()     #遍历右子树

def postOrder(self):                 #后序遍历
    if self.__left:
        self.__left.postOrder()
    if self.__right:
        self.__right.postOrder()
    print(self.__data)
```

二叉树

```
    def inOrder(self):                  #中序遍历
        if self.__left:
            self.__left.inOrder()
        print(self.__data)
        if self.__right:
            self.__right.inOrder()

if __name__ == '__main__':
    print('Please use me as a module.')
```

二叉树

■自定义二叉树用法

```
>>> import BinaryTree
>>> root = BinaryTree.BinaryTree('root')
>>> b = root.insertRightChild('B')
>>> a = root.insertLeftChild('A')
>>> c = a.insertLeftChild('C')
>>> d = c.insertRightChild('D')
>>> e = b.insertRightChild('E')
>>> f = e.insertLeftChild('F')
>>> root.inOrder()
C  D  A  root  B  F  E
>>> root.postOrder()
D  C  A  F  E  B  root
>>> b.inOrder()
B  F  E
```

有向图

```
def searchPath(graph, start, end):
    results = []
    __generatePath(graph, [start], end, results)
    results.sort(key = lambda x:len(x))
    return results

def __generatePath(graph, path, end, results):
    current = path[-1]
    if current == end:
        results.append(path)
    else:
        for n in graph[current]:
            if n not in path:
                #path.append(n)
                __generatePath(graph, path + [n], end, results)
```

有向图

```
def showPath(results):
    print('The path from ',results[0][0], ' to ', results[0][-1], ' is:')
    for path in results:
        print(path)
if __name__ == '__main__':
    graph = {'A':['B', 'C', 'D'],
                    'B':['E'],
                    'C':['D', 'F'],
                    'D':['B', 'E', 'G'],
                    'E':['D'],
                    'F':['D', 'G'],
                    'G':['E']}
    r1 = searchPath(graph, 'A', 'D')
    showPath(r1)
    r2 = searchPath(graph, 'A', 'E')
    showPath(r2)
```

总结

- 布尔值、if 条件判断
- for 循环、while 循环
- 列表推导式、字典推导式



第五部分 错误类型和异常捕获

- 错误类型
- 语法错误
- 异常
- 捕获异常
- assert、 with语句
- 自助控制异常
- 爬虫HTTP异常处理

错误类型

- 语法错误（Syntax errors）

  代码编译时的错误，不符合Python语言规则的代码会停止编译并返回错误信息

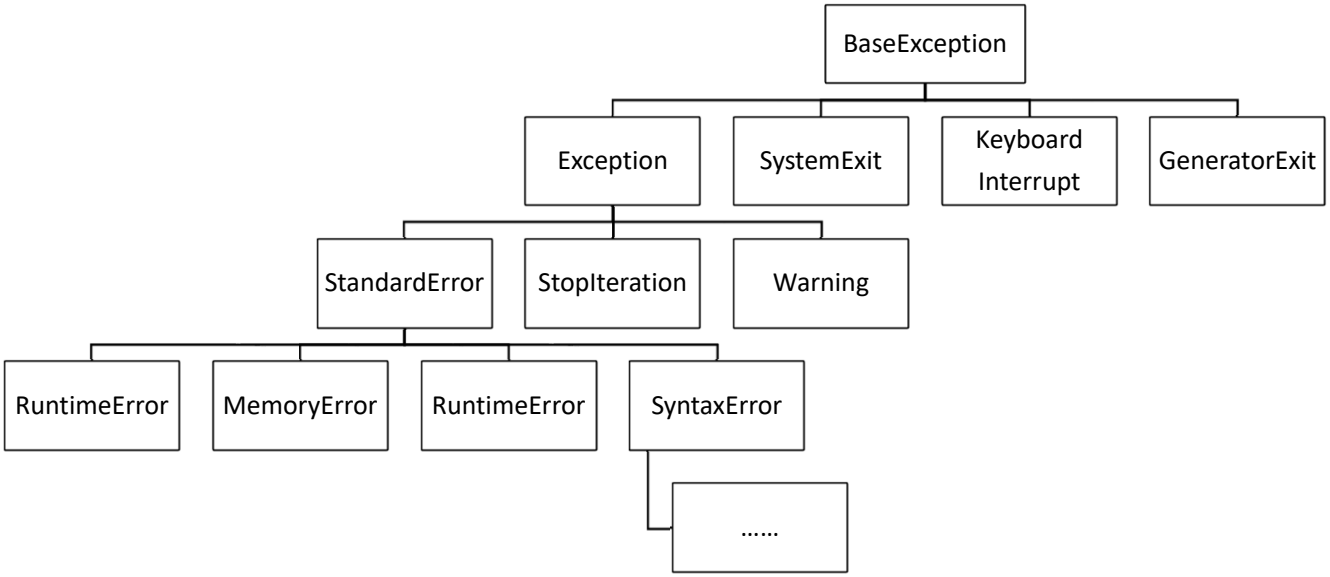
- 异常（Exceptions）

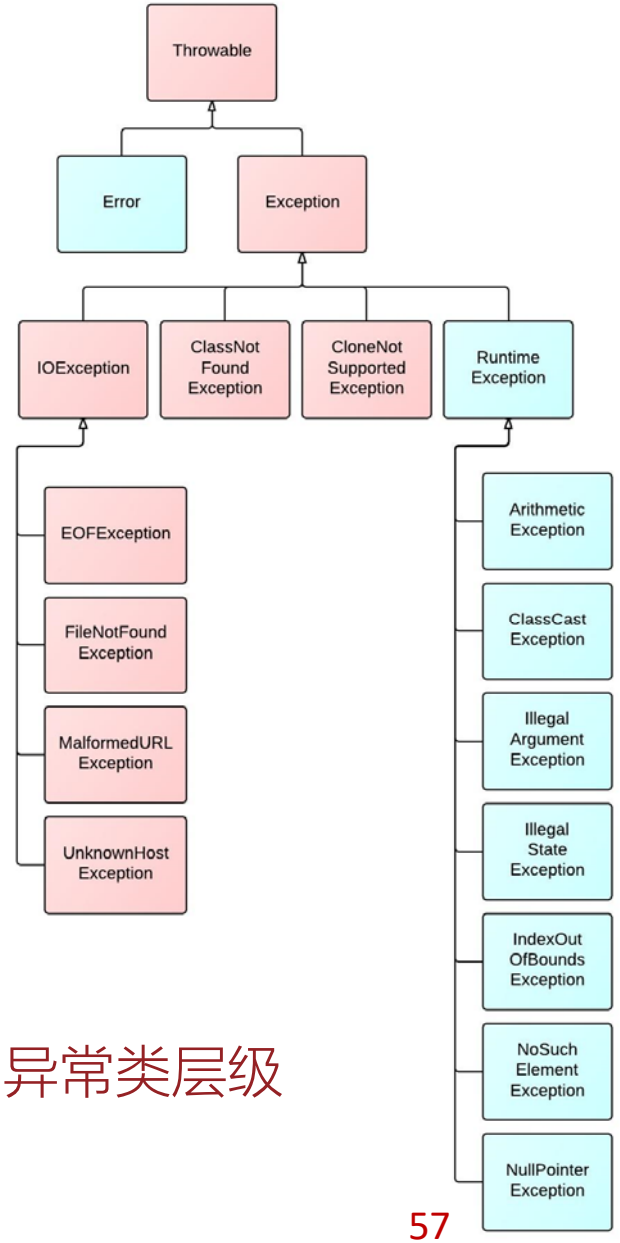
  相较于语法错误，异常比较难发现，因为它只在代码运行时才会发生，如类型错误、数值错误、索引错误和属性错误等。

- 语法错误包含在异常基类中

Python与Java异常类层级区别

Python异常类层级





Java异常类层级

语法错误

- 常见的语法错误（SyntaxError）包括：

- 缺少起始符号或结尾符号（括号、引号等）
- 缩进错误
- 关键词拼写错误

```
path = './Pokemon.csv
```

```
File "", line 1
  path = './Pokemon.csv
                       ^
SyntaxError: EOL while scanning string literal
```

语法错误

- 常见的语法错误（SyntaxError）包括：

- 缺少起始符号或结尾符号（括号、引号等）
- 缩进错误
- 关键词拼写错误

```
series = [1,2,3,4]
for num in series:
    print num
     print num
```

```
File "<ipython-input-14-9ba50fd15e27>", line 4
  print num
  ^
IndentationError: unexpected indent
```

语法错误

- 常见的语法错误（SyntaxError）包括：

- 缺少起始符号或结尾符号（括号、引号等）
- 缩进错误
- 关键词拼写错误

```
num = 3
prit num
```

```
  File "<ipython-input-22-6d2d92536390>", line 2
    prit num
         ^
SyntaxError: invalid syntax
```

异常

- 异常通常由以下问题引起：
  - 在定义函数之前就引用该函数
  - 调用不属于某个对象的方法或者属性
  - 试图将某个值转换为不恰当的数据类型

六种典型的异常

① 除零错误（ZeroDivisionError）：除数为0

② 名称错误（NameError）：变量使用前未进行申明或者初始化

```
num = 3
num/0
```

```
---------------------------------------------------------
ZeroDivisionError                         Traceback (m

<ipython-input-31-f24371e61331> in <module>()
      1 num = 3
----> 2 num/0

ZeroDivisionError: integer division or modulo by zero
```

```
variable_a
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)

<ipython-input-30-20d2ab2d3e10> in <module>()
----> 1 variable_a

NameError: name 'variable_a' is not defined
```

六种典型的异常

③ 类型错误（TypeError）：某些函数或者方法只适用于特定的数据类型，如果对数据类型的操作不当，就会产生类型错误

④ 数值错误（ValueError）：在输入类型正确的情况下，具体输入值错误

```
num = 3
num + '3'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recer

<ipython-input-21-804b0101d483> in <module>()
      1 num = 3
----> 2 num + '3'

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
float('3')
float('hackdata')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)

<ipython-input-23-8bd82f67428d> in <module>()
      1 float('3')
----> 2 float('hackdata')

ValueError: could not convert string to float: hackdata
```

六种典型的异常

⑤ 索引错误（IndexError）：超出序列长度的索引操作

⑥ 属性错误（AttributeError）：方法或者属性不适用该对象

```
list_hack = ['h','a','c','k']
list_hack[4]
```

```
---------------------------------------------

IndexError                                Trace

<ipython-input-25-92bac5b43ef3> in <module>()
      1 l = ['h','a','c','k']
----> 2 l[4]

IndexError: list index out of range
```

```
tuple_hack = ('h','a','c')
tuple.append('k')
```

```
---------------------------------------------------------------------------

AttributeError                            Traceback (most recent call last)

<ipython-input-26-38f0e0e728dc> in <module>()
      1 tuple_hack = ('h','a','c')
----> 2 tuple.append('k')

AttributeError: 'tuple' object has no attribute 'append'
```

异常层级

异常	描述
SyntaxError	语法错误
ZeroDivisionError	除数为0
NameError	尝试访问一个没有申明的变量
TypeError	对某种类型进行不恰当的操作
ValueError	传给函数的参数类型不正确，比如给 float() 函数传入字符串型变量
IndexError	索引超出序列范围
AttributeError	尝试访问未知的对象属性

```
BaseException
 +-- SystemExit
 +-- KeyboardInterrupt
 +-- GeneratorExit
 +-- Exception
      +-- StopIteration
      +-- StandardError
      |    +-- BufferError
      |    +-- ArithmeticError
      |    |    +-- FloatingPointError
      |    |    +-- OverflowError
      |    |    +-- ZeroDivisionError
      |    +-- AssertionError
      |    +-- AttributeError
      |    +-- EnvironmentError
      |    |    +-- IOError
      |    |    +-- OSError
      |    |         +-- WindowsError (Windows)
      |    |         +-- VMSError (VMS)
      |    +-- EOFError
      |    +-- ImportError
      |    +-- LookupError
      |    |    +-- IndexError
      |    |    +-- KeyError
      |    +-- MemoryError
      |    +-- NameError
      |    |    +-- UnboundLocalError
      |    +-- ReferenceError
      |    +-- RuntimeError
      |    |    +-- NotImplementedError
      |    +-- SyntaxError
      |    |    +-- IndentationError
      |    |         +-- TabError
      |    +-- SystemError
      |    +-- TypeError
      |    +-- ValueError
      |         +-- UnicodeError
      |              +-- UnicodeDecodeError
      |              +-- UnicodeEncodeError
      |              +-- UnicodeTranslateError
      +-- Warning
           +-- DeprecationWarning
           +-- PendingDeprecationWarning
           +-- RuntimeWarning
           +-- SyntaxWarning
           +-- UserWarning
           +-- FutureWarning
           +-- ImportWarning
```

捕获异常

- 代码编写环境自带的高亮显示
  - 便于发现常规语法错误
  - 但难于发现异常

Jupyter Notebook

In [ ]:
```
import csv #引入csv模块

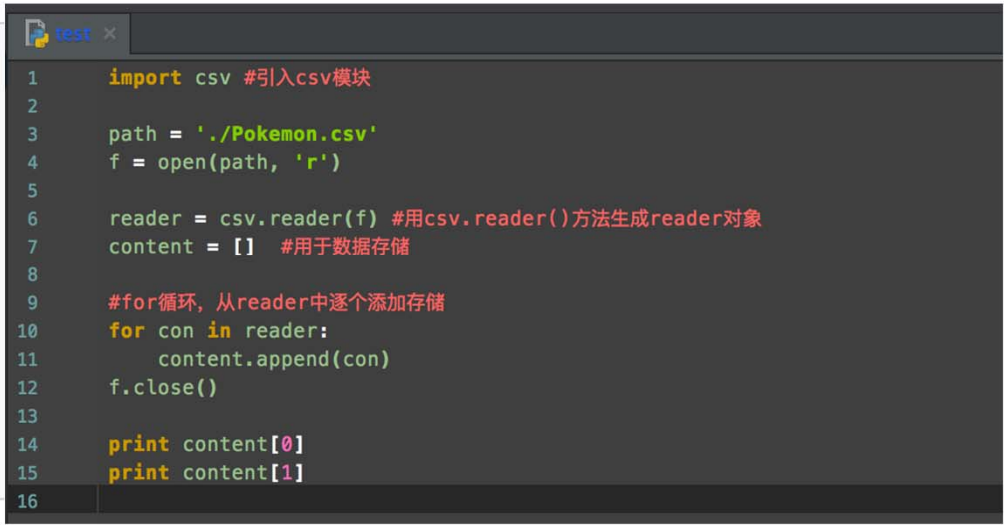
path = './Pokemon.csv'
f = open(path, 'r')

reader = csv.reader(f) #用csv.reader()方法生成reader对象
content = []  #用于数据存储

#for循环，从reader中逐个添加存储
for con in reader:
    content.append(con)
f.close()

print content[0]
print content[1]
```

PyCharm



捕获异常

- 程序要遇到异常的时候，往往是直接中断，跳出执行。但是有些时候，我们需要在遇到异常的时候另外处理，而不是直接停止。

- 解决方法：
  - try...except...语句
  - try...except...else语句
  - finally子句

try...except...语句

- try...except...语句

`try...except...` 是捕获异常最常用的语句，具体语法如下：

```
try:
    # 正常运行代码
except:
    # 不正常运行时的处理
```

- try 关键词内执行的是正常代码，当这部分代码出错的时候，会跳过错误代码后进入 except 关键词内部，执行此部分的代码

try...except...语句

```
short_list = [1, 72, 3]
position = 6
try:
    short_list[position]
except:
    print('索引应该在 0 和', len(short_list)-1,"之间,但却是",position)
```

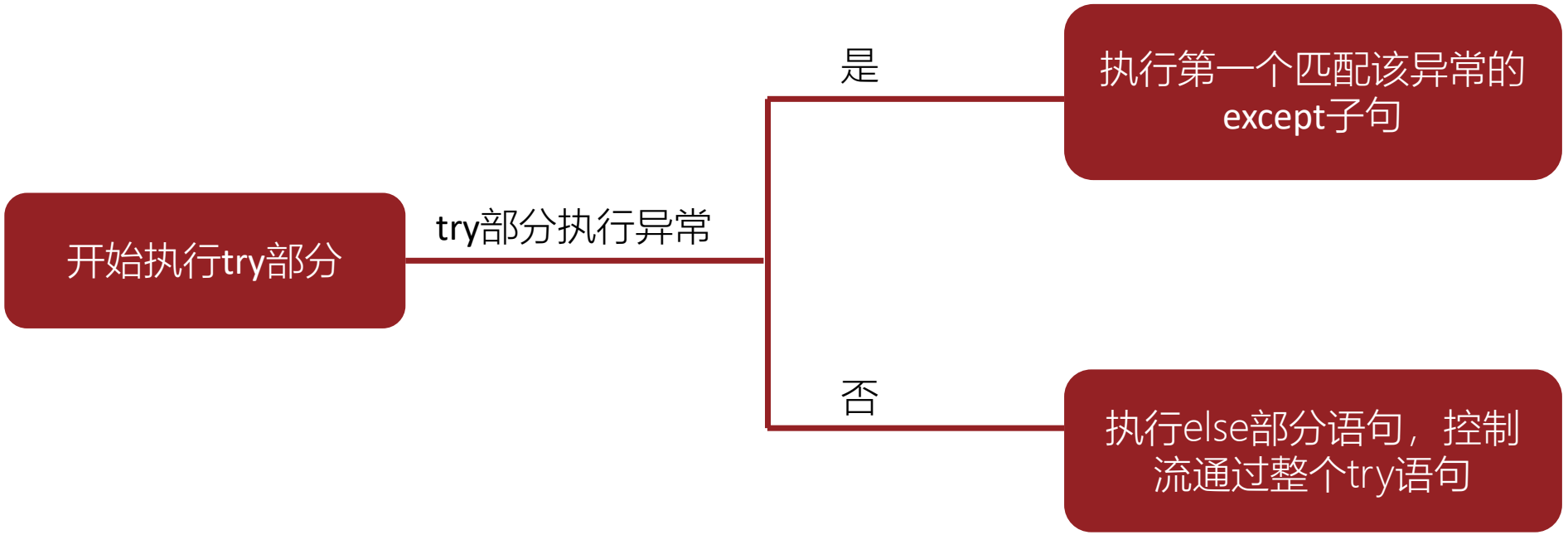
- 输出：
- 索引应该在 0 和 2 之间，但却是 6

try...except...else语句

- try...except...else语句
  - 当在 try...except... 后加入的 else 指，当程序没发生错误时执行的部分

```
try:
    # 正常运行
except(Exception1, Exception2, ...),e:
    # 发生Exception1, Exception2,...时的处理方式
else:
    # 正确时执行
```

- try...except...else...工作原理



- finally子句
  - finally 语句是指，无论程序运行对或错，都会执行的部分

```
try:
    # 正常运行
except(Exception1, Exception2, ...),e:
    # 发生Exception1, Exception2,...时的处理方式
else:
    # 正确时执行
finally:
    # 无论对错都执行
```

捕获异常

语法格式

```
try:
    语句块1
except 异常类型1:
    语句块2
except 异常类型2:
    语句块3
…
except 异常类型N:
    语句块N+1
except:
    语句块N+2
else:
    语句块N+3
finally:
   语句块N+4
```

捕获异常示例

除数为0的异常处理

```
x=int(input())
y=int(input())
try:
    result = x / y
except ZeroDivisionError:
    print("division by zero!")
else:
    print("result is", result)
finally:
    print("executing finally clause")
```

程序输入：

```
5
0
```

程序输出：

```
division by zero!
executing finally clause
```

assert、 with语句

- assert关键字
  - 当Expression部分为True时，则正确执行，程序继续下去；当判断为False时，则抛出后面的e错误提示。
  - 在大型的项目中，assert常被用来作为“防御性编程”

```
assert Expression, e
```

两种特殊的简便方法

- with语句
  - 有时候打开了文件却忘记关闭，或者是在读取文件过程出错，那么"with"语句能够很好解决关于文件读取、写入的问题

```
with open() as f:
    content = f.readlines()
```

- 上面的语句等价于

```
f=open()
content=f.readlines()#代码块
f.close()
```

自主控制异常：抛出异常

- 除了在代码运行出错时触发错误，我们还可以主动控制抛出异常，通过使用关键词 raise（类似Java语言中的throw）

```
raise ValueError('This is a value error!')
```

```
---------------------------------------------------------------------------
---
ValueError                                Traceback (most recent call la
st)
<ipython-input-6-4dc36a754212> in <module>()
----> 1 raise ValueError('This is a value error!')

ValueError: This is a value error!
```

自主控制异常：用户自定义异常

- 自定义异常的原因
  - Python提供的内建异常不够用
  - 可以预估某个错误的产生
- 定义异常类
  - 继承于Exception类，由它开始扩展

```
class MyError(Exception):
    pass

raise MyError(u'something error')
```

```
---------------------------------------------------------------------------
--
MyError                                   Traceback (most recent call las
t)
<ipython-input-4-d538cdc34e5a> in <module>()
      2     pass
      3
----> 4 raise MyError(u'something error')

MyError: something error
```

- 自主定义的NotIntError异常类，捕获非整型错误

```
class NotIntError(Exception):
    def __init__(self,error):
        self.error=error
a=[1,2,"",4,5,"a",['1','2','3']]
for i in range(len(a)):
    try:
        if type(a[i])!= int:
            raise NotIntError("Int 错误！")
    except NotIntError:
            print(NotIntError. with_traceback)
    finally:
            print(a[i])
```

以定义API异常为例

- 在编写API时，定义一个Root Exception根异常，其他异常都继承于根异常
  - 好处1：API代码层次更清晰
  - 好处2：API与调用程序代码隔离
- 如：需要做一个链接数据库服务的模块，提供一个connect函数用于链接，链接可能出现情况：
  - Socket连接超时
  - Socket拒绝连接

```
# database.py
class Error(Exception):
    '''Root exception for all
    exceptions raised by this module'''

class SocketTimeError(Error):
    pass

class SocketRefuseError(Error):
    pass

def connect():
    pass
```

调用API时异常捕获的技巧

- 这样精确定义多个异常，使得代码层次清晰，增强了可读性
- 在代码的最后还捕获了Error以及Exception两个异常，这两个操作分别对应于可拓展性与健壮性的目的

```
# 调用API
try:
    connect()
except SocketTimeError as err:
    log.error(err)
except SocketRefuseError as err:
    log.error(err)
except Error as err:
    log.error('API Unexpected error:%s' % err)
except Exception:
    log.error('API bug cause exception.')
```

使用异常代替返回状态码

- 编写工具类函数时，函数处理流程会产生很多状态
- 用返回值代表函数处理状态，调用者需要去理解每个状态码的意义，存在学习成本

```python
# 定义函数
def write(content):
    if isinstance(content, str):
        f = open("file.txt", 'w')
        try:
            f.write(content)
        except Exception:
            return -2     # 写入文件失败
        else:
            return 0      # 写入文件成功
        finally:
            f.close()
    else:
        return -1     # 输入类型错误
```

```python
# 调用函数
import logging # 打印日志信息
result = write(2)
if result == -1:
    logging.error(u"type error")
elif result == -2:
    logging.error(u"write error")
else:
    logging.info("ok")
```

```
ERROR:root:type error
```

- 使用异常的方式

```
# 定义异常
class Error(Exception):
    pass

class OpenFileError(Error):
    pass

class WriteContentError(Error):
    pass

def write(content):
    if isinstance(content, str):
        f = open("file.txt", 'w')
        try:
            f.write(content)
        except Exception:
            raise WriteContentError
        finally:
            f.close()
    else:
        raise OpenFileError
```

```
# 调用代码
import logging # 打印日志信息
try:
    write(2)
except OpenFileError as e:
    logging.error(e)
except WriteContentError as e:
    logging.error(e)
except Error:
    logging.error("API Error")
except Exception:
    logging.error("API Bug")
else:
    logging.info("ok")
```

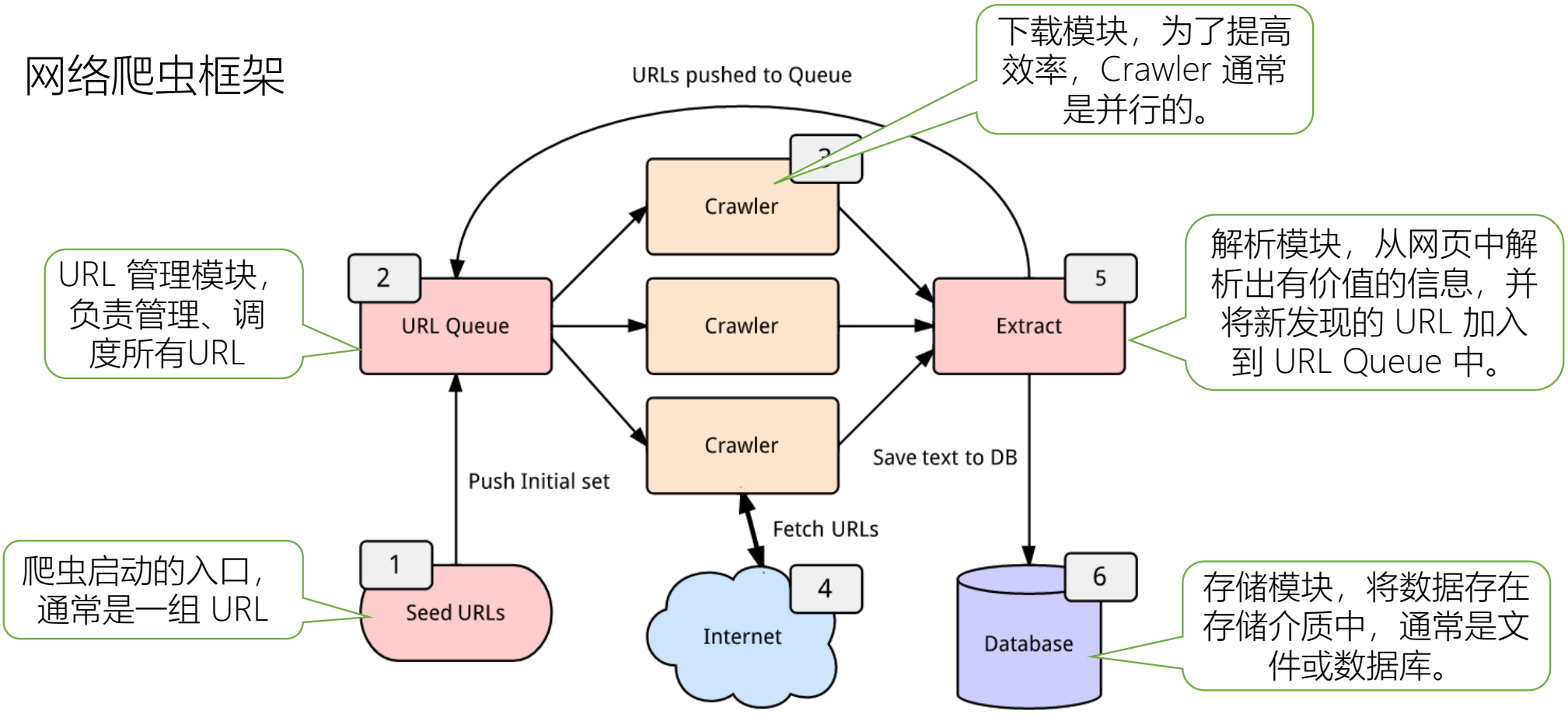
```
ERROR:root:
```

利用抛出异常并处理的优点

- 增强代码的鲁棒性而不必影响程序的主逻辑，专注程序主逻辑，保持代码的简洁清晰
- 异常判断是就地实时的，与代码运行是实时的
- 一些低调用级别的代码块没有判断和处理其上级调用模块的权限或者根本无法判断，只能由其上级调用模块来决定下一步怎么做，这时当前模块可以不用处理，将错误抛出给调用者

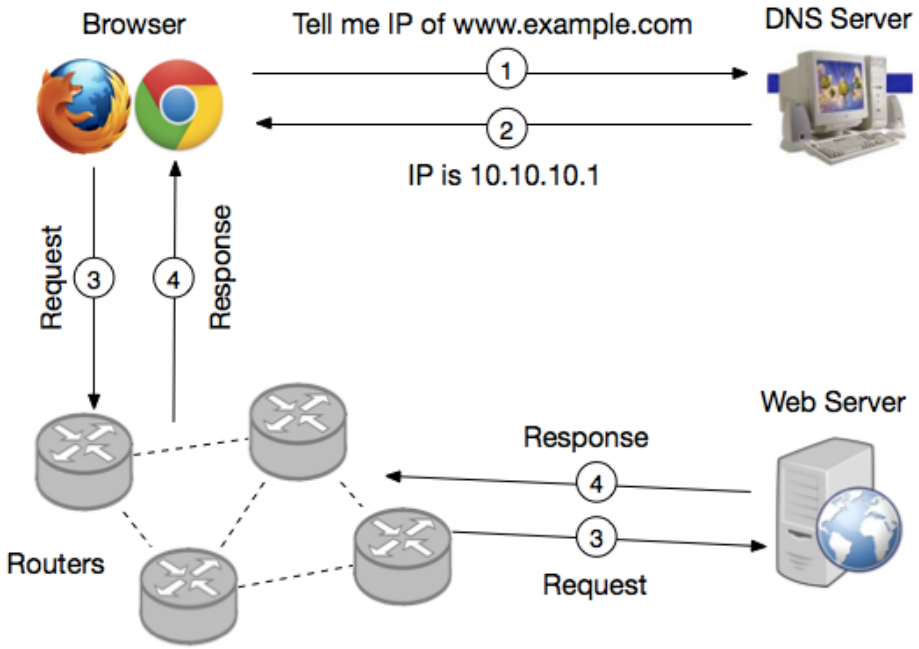
爬虫HTTP异常处理

- 网络爬虫框架



HTTP请求过程

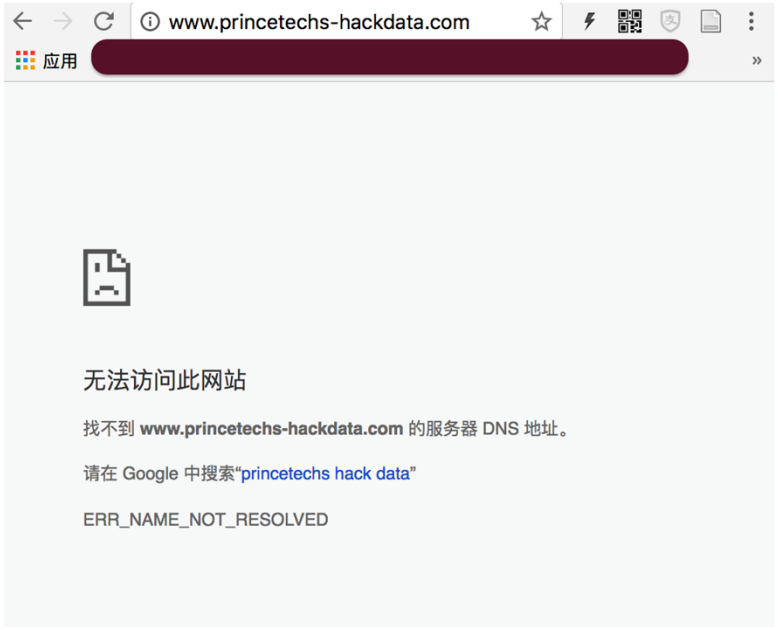
1. 获取要访问的 URL 的 IP 地址（右图中标号为1、2）
2. 向 Web Server 请求资源（右图中标号为3）
3. Web Server 收到请求，将响应返回给客户端（右图中标号为4）



HTTP请求过程的两种常见异常

① URLError：不能够处理一个response

- 网络无连接，即本机无法上网
- 连接不到特定的服务器
- 服务器不存在




```
import urllib2
request = urllib2.Request('http://www.princetechs-hackdata.com')
try:
    urllib2.urlopen(request)
except urllib2.URLError, e:
    print e.reason
```

```
[Errno 8] nodename nor servname provided, or not known
```

② HTTPError：URLError的子类，在特定的HTTP URLs中产生

```
import urllib2

request = urllib2.Request('http://www.hackdata.cn/study')
try:
    urllib2.urlopen(request)
except urllib2.HTTPError, e:
    print e.code
    print e.reason
```

```
404
NOT FOUND
```



- 每一个HTTP响应对象response包含一个数字“状态码”
- urllib2 会帮助处理重定向问题，不能处理则产生HTTPError
- 状态码：404-页面无法找到，403-请求禁止，401-带验证请求

异常处理技巧

① 捕获到HTTPError，输出code，不再处理URLError异常。如果发生的不是HTTPError，则去捕获URLError异常，输出错误原因

```
import urllib2

request = urllib2.Request('http://www.hackdata.cn/study')
try:
    urllib2.urlopen(request)
except urllib2.HTTPError, e:
    print e.code
except urllib2.URLError, e:
    print e.reason
else:
    print "OK"
```

```
404
```

② 利用 hasattr 方法提前对异常的属性进行判断，以免出现属性输出报错的现象

```
import urllib2

request = urllib2.Request('http://www.hackdata.cn/study')
try:
    urllib2.urlopen(request)
except urllib2.URLError, e:
    if hasattr(e,"code"):
        print e.code
    if hasattr(e,"reason"):
        print e.reason
else:
    print "OK"
```

```
404
NOT FOUND
```

一些经验

- 只处理你知道的异常，避免捕获所有异常然后吞掉它们
- 抛出的异常应该说明原因，有时候你知道异常类型也猜不出所以然的
- 不要使用异常来控制流程，那样你的程序会无比难懂和难维护
- 如果有需要，切记使用finally来释放资源

总结

- 语法错误和异常
- 捕获异常
- assert，with语句
- 抛出异常，自定义异常类
- 爬虫HTTP异常处理，URLError，HTTPError，code，reason